

Министерство науки и высшего образования Российской Федерации  
Федеральное государственное бюджетное образовательное учреждение  
высшего образования  
"Белгородский государственный технологический университет им. В. Г.  
Шухова"  
(БГТУ им. В.Г. Шухова)



Институт энергетики, информационных технологий и управляющих систем

Кафедра программного обеспечения вычислительной техники  
и автоматизированных систем

**Лабораторная работа № 7**  
**по дисциплине: Исследование операций**  
**по теме: Решение полностью целочисленных задач с помощью**  
**первого алгоритма Гомори, а также методом ветвей и границ**

Выполнил: студент группы ПВ-221  
Дорошенко Владимир Юрьевич  
Проверил: ст. преподаватель:  
Вирченко Ю.В.

**Белгород 2024**

**Цель работы:** Освоить метод отсечения Гомори для полностью целочисленных задач. Изучить алгоритм этого метода. Программно реализовать этот алгоритм.

### **Задания для подготовки к работе**

1. Изучить возможные постановки задач целочисленного и частично-целочисленного программирования.
2. Ознакомиться с методами решения таких задач, в частности, с методами отсечения и методом ветвей и границ.
3. Выяснить для каких задач применяется первый алгоритм Гомори. Изучить этот алгоритм и написать реализующую его программу для ПЭВМ. Изучить и программно реализовать алгоритм метода ветвей и границ. В качестве тестовых данных использовать, решенную вручную одну из нижеследующих задач.

7.

$$z = 10x_1 - 7x_2 \rightarrow \max;$$

$$\begin{cases} 21x_1 + 5x_2 + x_3 = 50, \\ 8x_1 - 5x_2 + 6x_4 = 13, \\ -7x_1 + x_2 + x_5 = 25, \end{cases}$$

$$x_i \geq 0, x_i - \text{целые } (i = \overline{1,5}).$$

Расширенная матрица системы ограничений-равенств данной задачи:

|    |   |   |   |   |    |
|----|---|---|---|---|----|
| 21 | 5 | 1 | 0 | 0 | 50 |
| 8  | 5 | 0 | 6 | 0 | 13 |
| -7 | 1 | 0 | 0 | 1 | 25 |

Приведем систему к единичной матрице методом жордановских преобразований.

1. В качестве базовой переменной можно выбрать  $x_3$ .

2. В качестве базовой переменной можно выбрать  $x_4$ .

Получаем новую матрицу:

|     |     |   |   |   |      |
|-----|-----|---|---|---|------|
| 21  | 5   | 1 | 0 | 0 | 50   |
| 4/3 | 5/6 | 0 | 1 | 0 | 13/6 |
| -7  | 1   | 0 | 0 | 1 | 25   |

3. В качестве базовой переменной можно выбрать  $x_5$ .

Поскольку в системе имеется единичная матрица, то в качестве базисных переменных принимаем  $X = (3, 4, 5)$ .

Выразим базисные переменные через остальные:

$$x_3 = -21x_1 - 5x_2 + 50$$

$$x_4 = -4/3x_1 - 5/6x_2 + 13/6$$

$$x_5 = 7x_1 - x_2 + 25$$

Подставим их в целевую функцию:

$$F(X) = 10x_1 + 7x_2$$

$$21x_1 + 5x_2 + x_3 = 50$$

$$4/3x_1 + 5/6x_2 + x_4 = 13/6$$

$$-7x_1 + x_2 + x_5 = 25$$

Матрица коэффициентов  $A = a(ij)$  этой системы уравнений имеет вид:

$A =$

|     |     |   |   |   |
|-----|-----|---|---|---|
| 21  | 5   | 1 | 0 | 0 |
| 4/3 | 5/6 | 0 | 1 | 0 |
| -7  | 1   | 0 | 0 | 1 |

| Базис    | B    | $x_1$ | $x_2$ | $x_3$ | $x_4$ | $x_5$ |
|----------|------|-------|-------|-------|-------|-------|
| $x_3$    | 50   | 21    | 5     | 1     | 0     | 0     |
| $x_4$    | 13/6 | 4/3   | 5/6   | 0     | 1     | 0     |
| $x_5$    | 25   | -7    | 1     | 0     | 0     | 1     |
| $F(x_0)$ | 0    | -10   | -7    | 0     | 0     | 0     |

**Итерация №0.**

| Базис          | B    | x <sub>1</sub> | x <sub>2</sub> | x <sub>3</sub> | x <sub>4</sub> | x <sub>5</sub> | min   |
|----------------|------|----------------|----------------|----------------|----------------|----------------|-------|
| x <sub>3</sub> | 50   | 21             | 5              | 1              | 0              | 0              | 50/21 |
| x <sub>4</sub> | 13/6 | 4/3            | 5/6            | 0              | 1              | 0              | 13/8  |
| x <sub>5</sub> | 25   | -7             | 1              | 0              | 0              | 1              | -     |
| F(x1)          | 0    | -10            | -7             | 0              | 0              | 0              | 0     |

#### 4. Пересчет симплекс-таблицы.

| B                        | x <sub>1</sub>            | x <sub>2</sub>           | x <sub>3</sub>        | x <sub>4</sub>        | x <sub>5</sub>        |
|--------------------------|---------------------------|--------------------------|-----------------------|-----------------------|-----------------------|
| $50 - (13/6 * 21) : 4/3$ | $21 - (4/3 * 21) : 4/3$   | $5 - (5/6 * 21) : 4/3$   | $1 - (0 * 21) : 4/3$  | $0 - (1 * 21) : 4/3$  | $0 - (0 * 21) : 4/3$  |
| $13/6 : 4/3$             | $4/3 : 4/3$               | $5/6 : 4/3$              | $0 : 4/3$             | $1 : 4/3$             | $0 : 4/3$             |
| $25 - (13/6 * -7) : 4/3$ | $-7 - (4/3 * -7) : 4/3$   | $1 - (5/6 * -7) : 4/3$   | $0 - (0 * -7) : 4/3$  | $0 - (1 * -7) : 4/3$  | $1 - (0 * -7) : 4/3$  |
| $0 - (13/6 * -10) : 4/3$ | $-10 - (4/3 * -10) : 4/3$ | $-7 - (5/6 * -10) : 4/3$ | $0 - (0 * -10) : 4/3$ | $0 - (1 * -10) : 4/3$ | $0 - (0 * -10) : 4/3$ |

Получаем новую симплекс-таблицу:

| Базис          | B     | x <sub>1</sub> | x <sub>2</sub> | x <sub>3</sub> | x <sub>4</sub> | x <sub>5</sub> |
|----------------|-------|----------------|----------------|----------------|----------------|----------------|
| x <sub>3</sub> | 127/8 | 0              | -65/8          | 1              | -63/4          | 0              |
| x <sub>1</sub> | 13/8  | 1              | 5/8            | 0              | 3/4            | 0              |
| x <sub>5</sub> | 291/8 | 0              | 43/8           | 0              | 21/4           | 1              |
| F(x1)          | 65/4  | 0              | -3/4           | 0              | 15/2           | 0              |

#### Итерация №1.

| Базис          | B     | x <sub>1</sub> | x <sub>2</sub> | x <sub>3</sub> | x <sub>4</sub> | x <sub>5</sub> | min    |
|----------------|-------|----------------|----------------|----------------|----------------|----------------|--------|
| x <sub>3</sub> | 127/8 | 0              | -65/8          | 1              | -63/4          | 0              | -      |
| x <sub>1</sub> | 13/8  | 1              | 5/8            | 0              | 3/4            | 0              | 13/5   |
| x <sub>5</sub> | 291/8 | 0              | 43/8           | 0              | 21/4           | 1              | 291/43 |
| F(x2)          | 65/4  | 0              | -3/4           | 0              | 15/2           | 0              | 0      |

#### 4. Пересчет симплекс-таблицы.

| B                             | $x_1$                  | $x_2$                        | $x_3$                  | $x_4$                        | $x_5$                  |
|-------------------------------|------------------------|------------------------------|------------------------|------------------------------|------------------------|
| $127/8 - (13/8 * 65/8) : 5/8$ | $0 - (1 * 65/8) : 5/8$ | $-65/8 - (5/8 * 65/8) : 5/8$ | $1 - (0 * 65/8) : 5/8$ | $-63/4 - (3/4 * 65/8) : 5/8$ | $0 - (0 * 65/8) : 5/8$ |
| $13/8 : 5/8$                  | $1 : 5/8$              | $5/8 : 5/8$                  | $0 : 5/8$              | $3/4 : 5/8$                  | $0 : 5/8$              |
| $291/8 - (13/8 * 43/8) : 5/8$ | $0 - (1 * 43/8) : 5/8$ | $43/8 - (5/8 * 43/8) : 5/8$  | $0 - (0 * 43/8) : 5/8$ | $21/4 - (3/4 * 43/8) : 5/8$  | $1 - (0 * 43/8) : 5/8$ |
| $65/4 - (13/8 * 3/4) : 5/8$   | $0 - (1 * 3/4) : 5/8$  | $-3/4 - (5/8 * 3/4) : 5/8$   | $0 - (0 * 3/4) : 5/8$  | $15/2 - (3/4 * 3/4) : 5/8$   | $0 - (0 * 3/4) : 5/8$  |

Получаем новую симплекс-таблицу:

| Базис      | B     | $x_1$ | $x_2$ | $x_3$ | $x_4$ | $x_5$ |
|------------|-------|-------|-------|-------|-------|-------|
| $x_3$      | 37    | 13    | 0     | 1     | -6    | 0     |
| $x_2$      | 13/5  | 8/5   | 1     | 0     | 6/5   | 0     |
| $x_5$      | 112/5 | -43/5 | 0     | 0     | -6/5  | 1     |
| F( $x_2$ ) | 91/5  | 6/5   | 0     | 0     | 42/5  | 0     |

#### 1. Проверка критерия оптимальности.

| Базис      | B     | $x_1$ | $x_2$ | $x_3$ | $x_4$ | $x_5$ |
|------------|-------|-------|-------|-------|-------|-------|
| $x_3$      | 37    | 13    | 0     | 1     | -6    | 0     |
| $x_2$      | 13/5  | 8/5   | 1     | 0     | 6/5   | 0     |
| $x_5$      | 112/5 | -43/5 | 0     | 0     | -6/5  | 1     |
| F( $x_3$ ) | 91/5  | 6/5   | 0     | 0     | 42/5  | 0     |

Оптимальный план можно записать так:

$$x_1 = 0, x_2 = 13/5, x_3 = 37, x_4 = 0, x_5 = 112/5$$

$$F(X) = 10 \cdot 0 + 7 \cdot 13/5 = 91/5$$

### Метод Гомори.

В полученном оптимальном плане присутствуют дробные числа.

По 2-у уравнению с переменной  $x_2$ , получившей нецелочисленное значение в оптимальном плане с наибольшей дробной частью  $\frac{3}{5}$ , составляем дополнительное ограничение:

$$q_2 - q_{21} \cdot x_1 - q_{22} \cdot x_2 - q_{23} \cdot x_3 - q_{24} \cdot x_4 - q_{25} \cdot x_5 \leq 0$$

$$q_2 = b_2 - [b_2] = \frac{13}{5} - 2 = \frac{3}{5}$$

$$q_{21} = a_{21} - [a_{21}] = \frac{8}{5} - 1 = \frac{3}{5}$$

$$q_{22} = a_{22} - [a_{22}] = 1 - 1 = 0$$

$$q_{23} = a_{23} - [a_{23}] = 0 - 0 = 0$$

$$q_{24} = a_{24} - [a_{24}] = \frac{6}{5} - 1 = \frac{1}{5}$$

$$q_{25} = a_{25} - [a_{25}] = 0 - 0 = 0$$

Дополнительное ограничение имеет вид:

$$\frac{3}{5} - \frac{3}{5}x_1 - \frac{1}{5}x_4 \leq 0$$

Преобразуем полученное неравенство в уравнение:

$$\frac{3}{5} - \frac{3}{5}x_1 - \frac{1}{5}x_4 + x_6 = 0$$

коэффициенты которого введем дополнительной строкой в оптимальную симплексную таблицу.

Поскольку двойственный симплекс-метод используется для поиска минимума целевой функции, делаем преобразование  $F(x) = -F(X)$ .

| Базис    | В               | $x_1$           | $x_2$ | $x_3$ | $x_4$           | $x_5$ | $x_6$ |
|----------|-----------------|-----------------|-------|-------|-----------------|-------|-------|
| $x_3$    | 37              | 13              | 0     | 1     | -6              | 0     | 0     |
| $x_2$    | $\frac{13}{5}$  | $\frac{8}{5}$   | 1     | 0     | $\frac{6}{5}$   | 0     | 0     |
| $x_5$    | $\frac{112}{5}$ | $-\frac{43}{5}$ | 0     | 0     | $-\frac{6}{5}$  | 1     | 0     |
| $x_6$    | $-\frac{3}{5}$  | $-\frac{3}{5}$  | 0     | 0     | $-\frac{1}{5}$  | 0     | 1     |
| $F(x_0)$ | $-\frac{91}{5}$ | $-\frac{6}{5}$  | 0     | 0     | $-\frac{42}{5}$ | 0     | 0     |

### 1. Проверка критерия оптимальности.

План 0 в симплексной таблице является псевдопланом, поэтому определяем ведущие строку и столбец.

### 2. Определение новой свободной переменной.

Среди отрицательных значений базисных переменных выбираем наибольший по модулю. Ведущей будет 4-ая строка, а переменную  $x_6$  следует вывести из базиса.

### 3. Определение новой базисной переменной.

Минимальное значение  $\theta$  соответствует 1-му столбцу, т.е. переменную  $x_1$  необходимо ввести в базис.

На пересечении ведущих строки и столбца находится разрешающий элемент (РЭ), равный  $(-\frac{3}{5})$ .

| Базис    | B     | $x_1$                               | $x_2$ | $x_3$ | $x_4$                                 | $x_5$ | $x_6$ |
|----------|-------|-------------------------------------|-------|-------|---------------------------------------|-------|-------|
| $x_3$    | 37    | 13                                  | 0     | 1     | -6                                    | 0     | 0     |
| $x_2$    | 13/5  | 8/5                                 | 1     | 0     | 6/5                                   | 0     | 0     |
| $x_5$    | 112/5 | -43/5                               | 0     | 0     | -6/5                                  | 1     | 0     |
| $x_6$    | -3/5  | -3/5                                | 0     | 0     | -1/5                                  | 0     | 1     |
| F(X0)    | -91/5 | -6/5                                | 0     | 0     | -42/5                                 | 0     | 0     |
| $\theta$ |       | $-\frac{6}{5} : (-\frac{3}{5}) = 2$ | -     | -     | $-\frac{42}{5} : (-\frac{1}{5}) = 42$ | -     | -     |

### 4. Пересчет симплекс-таблицы.

Выполняем преобразования симплексной таблицы методом Жордано-Гаусса.

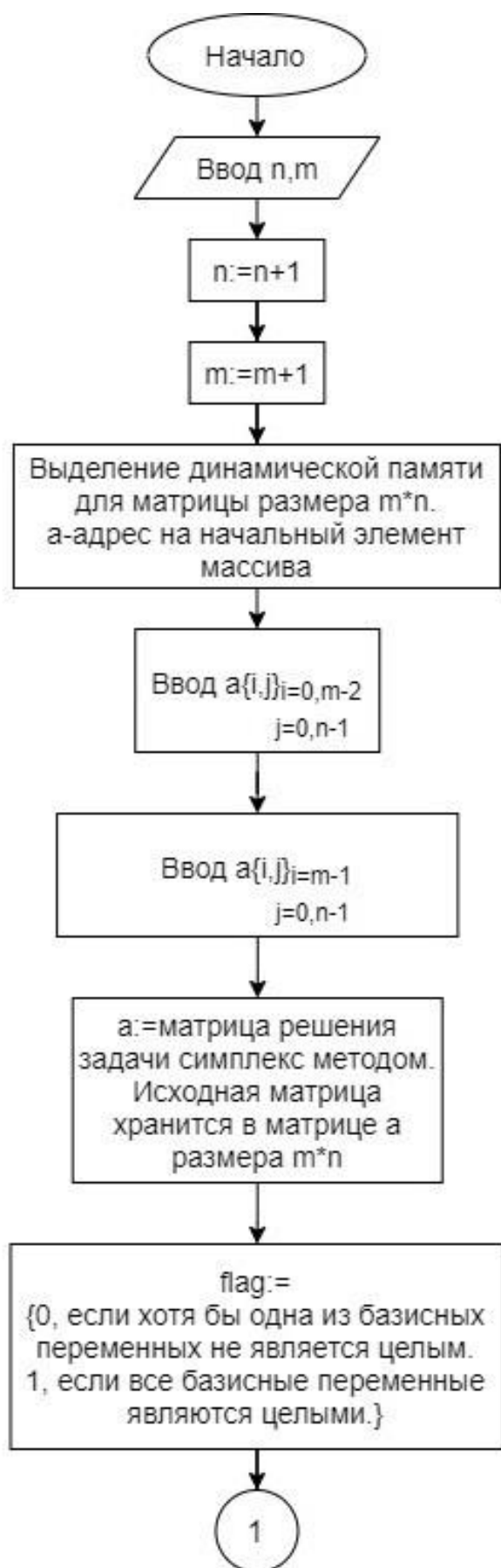
| Базис | B   | $x_1$ | $x_2$ | $x_3$ | $x_4$ | $x_5$ | $x_6$ |
|-------|-----|-------|-------|-------|-------|-------|-------|
| $x_3$ | 24  | 0     | 0     | 1     | -31/3 | 0     | 65/3  |
| $x_2$ | 1   | 0     | 1     | 0     | 2/3   | 0     | 8/3   |
| $x_5$ | 31  | 0     | 0     | 0     | 5/3   | 1     | -43/3 |
| $x_1$ | 1   | 1     | 0     | 0     | 1/3   | 0     | -5/3  |
| F(X0) | -17 | 0     | 0     | 0     | -8    | 0     | -2    |

Решение получилось целочисленным. Нет необходимости применять метод Гомори.

Оптимальный целочисленный план можно записать так:

$$x_1 = 1, x_2 = 1, x_3 = 24, x_4 = 0, x_5 = 31$$

$$F(X) = 10 \cdot 1 + 7 \cdot 1 = 17$$







Код программы:

```
include "simplex.h"

#define N 10

void simplex::simplexAlgorithm(Data * data, ostream & out) {

    printSimplexTable(out, data);

    int row, col;
    while (!checkIsOver(data)) {

        findLeadingElement(row, col, data);
        if (data->MFT[row] == INFINITY) data->unresolved = true;
        iterate(data, row, col);
        printSimplexTable(out, data);

    }
}

//returns the index of minimum in vector<>
int simplex::findMinInTheVector(const vector<float>& vec) {
    int index = 0;
    float min = vec[0];
    int size = vec.size();
    for (int i = 1; i < size; i++)
        if (vec[i] < min) {
            min = vec[i];
            index = i;
        }
    return index;
}

//finds element with min simplex coefficient
```

```

void simplex::findLeadingElement(int& row, int& col, Data* data) {

    col = findMinInTheVector(data->coefficients);
    fillMFT(col, data);
    row = findMinInTheVector(data->MFT);
    data->base[row] = col + 1;
}

//fills simplex coefficients vector
void simplex::fillMFT(int n, Data* data) {
    data->MFT.assign(data->system.size(), 0.0);
    for (int i = 0; i < data->MFT.size(); i++) {
        if (data->system[i][n] > 0)
            data->MFT[i] = data->freeMembers[i] / data->system[i][n];
        else data->MFT[i] = INFINITY;
    }
}

//fills simplex coefficients vector in dual simplex method
void simplex::dualFillMFT(int n, Data* data) {
    data->MFT.assign(data->coefficients.size(), 0.0);

    for (int i = 0; i < data->MFT.size(); i++) {
        if (data->system[n][i] < 0 && abs(data->coefficients[i]) > 0)
            data->MFT[i] = abs(data->coefficients[i]) / abs(data->system[n][i]);
        else data->MFT[i] = INFINITY;
    }
}

void simplex::iterate(Data* data, int row, int col) {

    int s = data->system.size();

```

```
int r = data->system[0].size();
```

```
float lead = data->system[row][col];
```

```
float curr;
```

```
//filling simplex matrix
```

```
for (int i = 0; i < s; i++) {
```

```
    curr = data->system[i][col];
```

```
    if (i != row) {
```

```
        data->freeMembers[i] -= data->freeMembers[row] * curr / lead;
```

```
        if (abs(data->freeMembers[i]) < 0.000001) data->freeMembers[i] = 0;
```

```
    }
```

```
    for (int j = 0; j < r; j++) {
```

```
        if (i == row) continue;
```

```
        else if (j == col) data->system[i][j] = 0;
```

```
        else data->system[i][j] -= (data->system[row][j] * curr / lead);
```

```
    }
```

```
}
```

```
//counting value of function
```

```
curr = data->coefficients[col];
```

```
data->max -= (data->freeMembers[row] / lead * curr);
```

```
data->freeMembers[row] /= lead;
```

```
//filling coefficients' row
```

```
for (int i = 0; i < r; i++) {
```

```
    data->coefficients[i] = (data->coefficients[i] - ((data->system[row][i] / lead) *  
curr));
```

```
}
```

```
//filling leading row
```

```
for (int i = 0; i < r; i++) {
```

```

        data->system[row][i] /= lead;
    }
}

// simplex algorithm isn't over if any of function coefficients are less than zero
bool simplex::checkIsOver(Data* data) {
    if (data->unresolved) return true;
    for (int i = 0; i < data->coefficients.size(); i++) {

        if (abs(data->coefficients[i]) < 0.000001) data->coefficients[i] = 0; // prevents
machine zero errors

        if (data->coefficients[i] < 0) return false;
    }
    return true;
}

//prints the results into readable form
void simplex::printSimplexTable(ostream& out, Data* data) {
    out.setf(ios::fixed);
    out.precision(3);

    int row = data->system.size();
    int col = data->system[0].size();

    out.width(N);
    out << "Base";
    out.width(N);
    out << "Plan";
    for (int j = 0; j < col; j++) {
        out.width(N - 1);
        out << "x";
    }
}

```

```

        out << j + 1;
    }

    out << endl;
    for (int i = 0; i < row; i++) {
        out.width(N - 1);
        out << "x";
        out << data->base[i];
        out.width(N);
        out << data->freeMembers[i];
        for (int j = 0; j < col; j++) {
            out.width(N);
            out << data->system[i][j];
        }
        out << endl;

    }

    out.width(N);
    out << "F";
    out.width(N);
    out << data->max;
    int a = data->coefficients.size();
    for (int i = 0; i < a; i++) {
        out.width(N);
        out << data->coefficients[i];
    }

    out << endl << endl;

}

```

```

void simplex::dualSimplexAlgorithm(Data* data, ostream& out) {

    printSimplexTable(out, data);
    int row, col;
    while (!dualCheckIsOver(data)) {
        dualFindLeadingElement(row, col, data);

        if (data->MFT[col] == INFINITY) data->unresolved = true;
        iterate(data, row, col);
        printSimplexTable(out, data);
    }
}

```

```

bool simplex::dualCheckIsOver(Data* data) {
    if (data->unresolved) return true;
    int s = data->freeMembers.size();
    for (int i = 0; i < s; i++) {
        if (data->freeMembers[i] < 0) return false;
    }
    return true;
}

```

```

void simplex::dualFindLeadingElement(int& row, int& col, Data* data) {

    row = findMinInTheVector(data->freeMembers);

    dualFillMFT(row, data);

    col = findMinInTheVector(data->MFT);
}

```

```

        data->base[row] = col + 1;
    }

int gomori::findMaxFractionalPart(vector<float> vec) {
    unsigned s = vec.size();
    int ind = 0;
    float max = fractionalPart(vec[0]);
    for (unsigned i = 1; i < s; i++) {
        if (fractionalPart(vec[i]) > max) {
            max = fractionalPart(vec[i]);
            ind = i;
        }
    }
    return ind;
}

```

```

float gomori::roundF(float n) {
    return n + 0.00005f;
}

```

```

float gomori::fractionalPart(float n) {
    if (abs(n) < 0.00001) return 0;

    if (n > 0) return (n - (int)roundF(n));
    if (fractionalPart(abs(n)) < 0.000001f) return 0;
    else return (n - (int)roundF(n - 1));
}

```

```

bool gomori::checkIsOver(Data* data) {

```



```

        unsigned s = data->freeMembers.size();
        for (unsigned i = 0; i < s; i++) {

            if (fractionalPart(data->freeMembers[i]) > 0.0001)
                return false;
        }
        return true;
    }

    // adds new restriction into simplex system
    // the restriction is based on followed line with index n
    void gomori::addRestriction(int n, Data* data) {

        data->freeMembers.push_back(-fractionalPart(data->freeMembers[n]));

        vector<float> newRow;
        for (unsigned i = 0; i < data->system[n].size(); i++)
            newRow.push_back(-fractionalPart(data->system[n][i]));

        //adding new element into base
        newRow.push_back(1);
        data->base.push_back(newRow.size()); // stores the number of base element

        //resizing of simplex system
        data->coefficients.push_back(0);
        for (unsigned i = 0; i < data->system.size(); i++)
            data->system[i].push_back(0);
        data->system.push_back(newRow);
    }

    void gomori::gomoriAlgorithm(Data* data, ostream& out) {
        simplex::simplexAlhorithm(data, out);
    }

```

```

while (!checkIsOver(data)) {
    int n = findMaxFractionalPart(data->freeMembers);
    addRestriction(n, data);
    simplex::dualSimplexAlgorithm(data, out);
}

}

```

```

#pragma once

```

```

#include <sstream>

```

```

#include "data.h"

```

```

class simplex {
    static void printSimplexTable(ostream&, Data*);
    static int findMinInTheVector(const vector<float>&);
    static void iterate(Data*, int, int);

    static void fillMFT(int, Data*);
    static bool checkIsOver(Data*);
    static void findLeadingElement(int&, int&, Data*);

    static void dualFillMFT(int, Data*);
    static bool dualCheckIsOver(Data*);
    static void dualFindLeadingElement(int&, int&, Data*);
public:

```

```

static void simplexAlgorithm(Data*, ostream&);

static void dualSimplexAlgorithm(Data*, ostream&);

};

class gomori {

    static int findMaxFractionalPart(vector<float>);
    static int findMaxFractionalPart_2(vector<float>, Data*);
    static inline float fractionalPart(float n);
    static void addRestriction(int, Data*);
    static void addRestriction_2(int, Data*);
    static inline float roundF(float);
    static bool checkIsOver(Data*);
    static bool checkIsOver_2(Data*);
public:
    static void gomoriAlgorithm(Data*, ostream&);

};

```

```

#include "data.h"

```

```

#define N 10

```

```

using std::string;

```

```

inline bool isNumber(char ch) {

```

```

        if (ch <= '9' && ch >= '0') return true;

        return false;
    }

    //gets coefficients from string
    void Data::getCoefficients(string func, vector<float>& coeff)
    {
        char ch = 0;
        float curr = 0;
        unsigned length = func.length();
        bool isNeg = false;
        for (unsigned i = 0; i < length; i++) {
            if (func[i] == 'x' && !isNumber(func[i + 1]))continue;
            if (func[i] == '-') isNeg = true;

            if (isNumber(func[i]) || func[i] == 'x') {

                string coef;
                if (func[i] == 'x') coef = "1";
                else {
                    coef += func[i];
                    while (func[++i] != 'x') {
                        coef += func[i];
                    }
                }

                if (isNeg) {
                    coef = "-" + coef;
                    isNeg = false;
                }
            }
        }
    }

```

```

        string num;

        num += func[++i];
        while (isNumber(func[++i])) {
            num += func[i];
        }

        int curVar = stoi(num);
        while (coeff.size() + 1 < curVar) {

            coeff.push_back(0.0);
        }
        curr = stof(coef);
        maxCoef = (curr > maxCoef) ? curr : maxCoef;
        coeff.push_back(curr);
    }
}

while (coefficients.size() > coeff.size()) coeff.push_back(0.0);
}

//reads information from followed stream
// information must be structured in such form:
// function       $f(x) = ax_1 + bx_2 + cx_3 + \dots$  , where a, b, c... - function's coefficients
// mode          (max or min)
// system of restrictions  $a_{11}x_1 + a_{12}x_2 + \dots + a_{1n}x_n \leq b_1$ 
//                                      $a_{21}x_1 + a_{22}x_2 + \dots + a_{2n}x_n \leq b_1$ 
//                                     .....
//                                      $a_{m1}x_1 + a_{m2}x_2 + \dots + a_{mn}x_n \leq b_m$ 
//
//                                     END

// instead of " $\leq$ " can be used " $\geq$ " or "="
void Data::readUserData(std::istream& in) {

```

```

//
string function;

getline(in, function);

getCoefficients(function, coefficients);


string mode;

getline(in, mode);

if (mode == "min") isMaximization = false;
else if (mode == "max") isMaximization = true;


string st = "";

getline(in, st);

//reading system of restrictions
while (!(in.eof() || st == "END")) {
    string num;

    int i = st.length();

    //the free member separation
    while (isNumber(st[--i]) || st[i] == '.');

    num = st.substr(i + 1, st.length());

    float n = stof(num);

    while (st[i] != '=') {
        if (st[i] == '-') n = -n;

        i--;
    }

    freeMembers.push_back(n);

    //processing sign
    if (st[i - 1] == '<') signOfRestrictions.push_back(1);
    else if (st[i - 1] == '>') signOfRestrictions.push_back(-1);
    else signOfRestrictions.push_back(0);
}

```

```

        vector<float> vect;

        getCoefficients(st.substr(0, i), vect);

        system.push_back(vect);

        getline(in, st);

    }

    maxCoef *= 2;

    convertation();

}

//converts information after reading into standart form
void Data::convertation() {
    // standart mode - minimization
    if (isMaximization) {
        isMaximization = false;

        int s = coefficients.size();

        for (int i = 0; i < s; i++) {
            coefficients[i] = -coefficients[i];
        }
    }

    //standart sign - "<="
    //also here an artificial base is created
    int size = signOfRestrictions.size();
    for (int i = 0; i < size; i++) {
        if (signOfRestrictions[i] == 0) {
            coefficients.push_back(maxCoef);
            base.push_back(i);

            int s = system.size();
            for (int j = 0; j < s; j++) {

```

```

        if (j != i) system[j].push_back(0);
        else {
            system[j].push_back(1);
            base.push_back(coefficients.size());
        }
    }
}

else if (signOfRestrictions[i] > 0) {
    coefficients.push_back(0);
    coefficients.push_back(maxCoef);
    int s = system.size();
    for (int j = 0; j < s; j++) {
        if (j != i) {
            system[j].push_back(0);
            system[j].push_back(0);
        }
        else {
            system[j].push_back(1);
            system[j].push_back(1);
            base.push_back(coefficients.size());
        }
    }
}

else {
    coefficients.push_back(0);
    coefficients.push_back(maxCoef);
    int s = system.size();
    for (int j = 0; j < s; j++) {
        if (j != i) {

```



```

        system[j].push_back(0);
        system[j].push_back(0);
    }
    else {
        system[j].push_back(-1);
        system[j].push_back(1);
        base.push_back(coefficients.size());
    }
}
}
}

```

//we need the sum of elements in each column

```
int s = coefficients.size();
```

```
float free = 0; //the sum of free members
```

```
for (int i = 0; i < freeMembers.size(); i++)
```

```
    free += freeMembers[i];
```

```
float* sum = new float[s];
```

```
for (int i = 0; i < s; i++)
```

```
    sum[i] = 0;
```

```
for (int i = 0; i < system.size(); i++) {
```

```
    for (int j = 0; j < s; j++) {
```

```
        sum[j] += system[i][j];
```

```
    }
```

```
}
```

// after creating an artificial base we need to multiply coefficients

```
for (int i = 0; i < s; i++)
```

```
    sum[i] *= maxCoef;
```

```

    free *= maxCoef;

    for (int i = 0; i < s; i++)
        coefficients[i] -= sum[i];

    max = -free; // the function value in this particular point
    MFT.assign(system.size(), 0.0); // vector of simplex coefficients
}

```

```

#pragma once

#include <iostream>
#include <string>
#include <vector>
#include <set>
using namespace std;

class Data {
    friend class gomori;
    friend class simplex;
public:

    void readUserData(std::istream& in);
    void getCoefficients(string, vector<float>&);
    void convertation();

protected:
    bool isMaximization;
    bool isMixed = false;

```

```

float max = 0, maxCoef = 0;

vector<float> MFT;

set<int> notIntegers;

vector<int> base;

vector<float> coefficients;


//free members in restrictions

vector<float> freeMembers;


//matrix of coefficients of restrictions

vector<vector<float>>> system;


//sign for each restriction ["=" -> 0, "<=" -> 1, ">=" -> -1]

vector<int> signOfRestrictions;

bool unresolved = false;

};

int main(int argc, char** argv) {

    ifstream in;

    in.open("input.txt");

    Data d;

    d.readUserData(in);

    in.close();

    ofstream out;

    out.open("output.txt");

    gomori::gomoriAlgorithm(&d, out);

    out.close();

    system("pause");

}

```

```

Введите кол-во уравнений: 3
Введите кол-во независимых переменных: 5
Введите систему уравнений, в каноничном виде, в виде матрицы:
9 3 1 0 0 18
3 -5 0 4 0 10
-2 3 0 0 1 5
Введите уравнение z в виде строки матрицы:
2 0 1 0 0 0
Введённая задача:

```

$z = 2.00x_1 + x_3 \rightarrow \max$

```

| 9.00x_1 + 3.00x_2 + x_3 = 18
| 3.00x_1 - 5.00x_2 + 4.00x_4 = 10
| -2.00x_1 + 3.00x_2 + x_5 = 5
|

```

Таблица после изменения строки:

| Баз-е пер-е | Своб. коэф. | x1   | x2   | x3   | x4   | x5   | u1   | u2    | u3    |
|-------------|-------------|------|------|------|------|------|------|-------|-------|
| x3          | 6.00        | 0.00 | 0.00 | 1.00 | 0.00 | 0.00 | 0.00 | 12.00 | 6.00  |
| x4          | 3.00        | 0.00 | 0.00 | 0.00 | 1.00 | 0.00 | 0.00 | -0.50 | 2.00  |
| x1          | 1.00        | 1.00 | 0.00 | 0.00 | 0.00 | 0.00 | 0.00 | -1.00 | -1.00 |
| x2          | 1.00        | 0.00 | 1.00 | 0.00 | 0.00 | 0.00 | 0.00 | -1.00 | 1.00  |
| u1          | 1.00        | 0.00 | 0.00 | 0.00 | 0.00 | 0.00 | 1.00 | -1.50 | 0.00  |
| x5          | 4.00        | 0.00 | 0.00 | 0.00 | 0.00 | 1.00 | 0.00 | 1.00  | -5.00 |
| z           | 8.00        | 0.00 | 0.00 | 0.00 | 0.00 | 0.00 | 0.00 | 10.00 | 4.00  |

Последняя таблица лишена дробных чисел, поэтому она является итоговой.

$z_{\max} = 8$

$x_3 = 6$

$x_4 = 3$

$x_1 = 1$

$x_2 = 1$

$u_1 = 1$

$x_5 = 4$

## Вывод

После освоения метода отсечения Гомори для полностью целочисленных задач и его программной реализации можно сделать следующие выводы:

Метод отсечения Гомори представляет собой мощный инструмент для решения задач линейного программирования с целочисленными ограничениями. Его основная идея заключается в построении сепараторов, которые исключают недопустимые целочисленные решения из области поиска. Программная реализация этого метода требует умения работать с линейным программированием и эффективного использования библиотек для работы с матрицами и оптимизации.

Использование метода отсечения Гомори может существенно улучшить процесс решения целочисленных задач линейного программирования, особенно в случаях, когда классические методы неэффективны или не применимы.